



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

PROGRAMOWANIE OBIEKTOWE 2

*Dokumentacja do projektu C# na temat: System
zarządzania hotelem*

Dmytro Marienko

Nr albumu: 131469

Kierunek studiów: Informatyka

Prowadzący zajęcia laboratoryjne: mgr inż. Ewa Żesławska

Rzeszów, 2026

Streszczenie w języku polskim

Celem projektu było wykonanie aplikacji desktopowej do zarządzania hotelem. Program ma pomagać pracownikowi recepcji w kontroli podstawowych danych, takich jak goście, pokoje, typy pokoi, rezerwacje, płatności, pracownicy oraz usługi dodatkowe.

W projekcie zastosowano język C#, technologię WPF do wykonania interfejsu użytkownika oraz Entity Framework Core do obsługi lokalnej bazy danych SQLite. Aplikacja posiada panel główny ze statystykami, zakładki nawigacyjne, formularze do dodawania danych, tabele z rekordami oraz podstawową walidację pól.

Główne funkcjonalności programu obejmują dodawanie gości, tworzenie rezerwacji, sprawdzanie dostępności pokoi, rejestrowanie płatności oraz dodawanie i edycję usług hotelowych. Końcowym efektem projektu jest działająca aplikacja, która realizuje najważniejsze założenia prostego systemu zarządzania hotelem.

Abstract in English

The aim of the project was to create a desktop application for hotel management. The program helps a reception employee manage basic data such as guests, rooms, room types, reservations, payments, employees and additional hotel services.

The project uses C#, WPF for the user interface and Entity Framework Core for access to the local SQLite database. The application contains a dashboard with statistics, navigation tabs, forms for entering data, tables with records and basic field validation.

The main functions of the program include adding guests, creating reservations, checking room availability, registering payments and adding or editing hotel services. The final result is a working application that implements the main assumptions of a simple hotel management system.

Spis treści

1. Wprowadzenie i analiza problemu	9
1.1. Opis analizowanego problemu.....	9
1.2. Cel projektu	9
1.3. Zakres realizowanych funkcjonalności.....	9
1.4. Przegląd podobnych rozwiązań	10
2. Wymagania systemowe	11
2.1. Wymagania funkcjonalne aplikacji.....	11
2.2. Wymagania нефункционалне.....	11
2.3. Założenia dotyczące działania systemu	11
3. Architektura i struktura aplikacji.....	13
3.1. Architektura aplikacji	13
3.2. Podział projektu na warstwy i moduły	13
3.3. Struktura folderów oraz organizacja kodu	13
3.4. Zastosowane wzorce projektowe	14
3.5. Mechanizmy walidacji danych	14
3.6. Obsługa błędów i wyjątków	14
3.7. Czytelność i organizacja projektu.....	15
4. Technologie wykorzystane w projekcie	16
4.1. Technologie interfejsu użytkownika	16
4.2. Technologie dostępu do danych.....	16
4.3. Dodatkowe biblioteki i narzędzia	16
4.4. Uzasadnienie wyboru technologii.....	16
5. Projekt bazy danych	17
5.1. Opis modelu danych	17
5.2. Diagram ERD	17
5.3. Opis relacji i kluczy	18
5.4. Informacje dotyczące normalizacji.....	18
5.5. Opis migracji bazy danych	18
5.6. Mechanizmy zapewniające integralność danych	19
6. Implementacja aplikacji.....	20
6.1. Najważniejsze moduły systemu.....	20
6.2. Logika działania aplikacji.....	20
6.3. Realizacja operacji CRUD.....	20
6.4. Dodawanie i edycja usług hotelowych	21
6.5. Mechanizmy logowania i autoryzacji	22
6.6. Organizacja nawigacji.....	22

6.7. Interfejs użytkownika	22
6.8. Stylizacja i responsywność aplikacji	22
6.9. Walidacja danych w kodzie	22
7. Prezentacja warstwy użytkowej projektu.....	24
7.1. Zrzuty ekranu aplikacji	24
7.2. Opis działania najważniejszych funkcjonalności	27
7.3. Prezentacja interfejsu użytkownika	28
7.4. Przykładowe scenariusze działania aplikacji.....	28
8. Harmonogram realizacji projektu	29
9. Instrukcja uruchomienia projektu	30
9.1. Wymagania systemowe.....	30
9.2. Sposób konfiguracji środowiska	30
9.3. Instrukcja uruchomienia aplikacji.....	30
9.4. Informacje dotyczące bazy danych.....	30
9.5. Dane logowania testowego	31
9.6. Miejsce na repozytorium GitHub	31
10. Podsumowanie	32
10.1. Ocena stopnia realizacji założeń projektowych.....	32
10.2. Napotkane problemy.....	32
10.3. Możliwe kierunki dalszego rozwoju aplikacji	32
Spis rysunków	33
Spis tabel	34
Spis listingów	35

1. Wprowadzenie i analiza problemu

1.1. Opis analizowanego problemu

Hotel przechowuje wiele informacji, które są ze sobą powiązane. Pracownik recepcji musi znać dane gości, dostępność pokoi, terminy rezerwacji, płatności oraz usługi dodatkowe. Jeżeli takie informacje są prowadzone tylko w notatkach albo arkuszu kalkulacyjnym, łatwo o pomyłkę, na przykład podwójną rezerwację tego samego pokoju w tym samym terminie.

Analizowany problem dotyczy podstawowego zarządzania hotelem w aplikacji desktopowej. Projekt nie jest pełnym systemem hotelowym klasy komercyjnej. Nie zawiera modułu płatności online, integracji z portalami rezerwacyjnymi ani logowania wielu użytkowników. Zakres został dobrany tak, aby program był możliwy do wykonania jako projekt studencki, ale jednocześnie miał rozbudowaną bazę danych i kilka powiązanych modułów.

1.2. Cel projektu

Celem projektu było przygotowanie aplikacji desktopowej w języku C#, która umożliwi podstawową obsługę hotelu. System ma pozwalać na przeglądanie danych hotelowych, dodawanie gości, tworzenie rezerwacji, sprawdzanie dostępności pokoi, rejestrowanie płatności oraz zarządzanie usługami dodatkowymi.

1.3. Zakres realizowanych funkcjonalności

W projekcie zrealizowano podstawowy zakres funkcjonalności potrzebny do pracy recepcji hotelowej. Użytkownik może:

- przeglądać dane w tabelach,
- dodawać nowych gości hotelowych,
- przeglądać pokoje oraz typy pokoi,
- tworzyć rezerwacje dla wybranego gościa i pokoju,
- sprawdzać dostępność pokoi w podanym terminie,
- rejestrować płatności dla wybranej rezerwacji,
- przeglądać listę pracowników,
- dodawać nowe usługi hotelowe,
- edytować istniejące usługi hotelowe,
- sprawdzać szybkie statystyki na panelu głównym,
- poruszać się po aplikacji za pomocą zakładek.

1.4. Przegląd podobnych rozwiązań

Pierwszym podobnym rozwiązaniem jest Booking.com. Jest to duża platforma internetowa do wyszukiwania i rezerwowania noclegów. W porównaniu z takim rozwiązaniem mój projekt nie obsługuje klientów przez Internet, tylko skupia się na lokalnej pracy recepcji.

Drugim przykładem jest system KWHotel. Jest to program przeznaczony do zarządzania obiektami noclegowymi, rezerwacjami i pokojami. Mój projekt realizuje podobny ogólny pomysł, ale w prostszym zakresie i bez funkcji biznesowych, takich jak faktury czy synchronizacja z portalami.

Trzecim przykładem jest Cloudbeds albo inny system PMS dla hoteli. Takie systemy są rozbudowane i często działają w chmurze. Mój projekt został wykonany jako lokalna aplikacja desktopowa, dlatego jest prostszy, ale zawiera najważniejsze elementy: pokoje, rezerwacje, gości, płatności i usługi.

2. Wymagania systemowe

2.1. Wymagania funkcjonalne aplikacji

Aplikacja powinna umożliwiać podstawową obsługę danych hotelowych. System nie jest pełnym programem komercyjnym, tylko prostą aplikacją do zarządzania najważniejszymi elementami hotelu.

Tabela 2.1. Najważniejsze wymagania funkcjonalne aplikacji

Moduł	Wymaganie
Goście	Dodawanie nowych gości oraz wyświetlanie ich w tabeli.
Pokoje	Przeglądanie pokoi, numerów, statusów oraz typów pokoi.
Rezerwacje	Tworzenie rezerwacji dla wybranego gościa i pokoju.
Dostępność	Sprawdzanie, które pokoje są dostępne w wybranym terminie.
Płatności	Dodawanie płatności do wybranej rezerwacji.
Usługi	Dodawanie i edytowanie usług dodatkowych.
Panel główny	Wyświetlanie statystyk, liczby gości, rezerwacji i płatności.

Do wymagań funkcjonalnych należą także walidacja podstawowych pól, odświeżanie danych po wykonaniu operacji oraz pokazywanie komunikatów o błędach w dolnej części okna.

2.2. Wymagania niefunkcjonalne

Aplikacja powinna działać lokalnie na komputerze z systemem Windows. Interfejs powinien być czytelny i spójny, aby użytkownik mógł łatwo przechodzić między modułami. Kod powinien być podzielony na warstwy i pliki według odpowiedzialności.

Do wymagań niefunkcjonalnych należą:

- lokalne działanie programu na komputerze z systemem Windows,
- zapis danych w lokalnej bazie SQLite,
- czytelny układ głównego okna,
- podział projektu na modele, kontekst bazy, repozytoria, serwisy i ViewModel,
- podstawowa obsługa wyjątków,
- walidacja danych wpisywanych w formularzach,
- użycie Entity Framework Core do dostępu do danych,
- możliwość łatwego uruchomienia projektu w Visual Studio 2022.

2.3. Założenia dotyczące działania systemu

Założono, że program działa lokalnie i korzysta z jednej bazy danych zapisanej w pliku `hotel.db`. Użytkownik nie musi zakładać konta i nie musi się logować. Dane testowe są dodawane automatycznie przy pierwszym uruchomieniu aplikacji.

System służy do podstawowego zarządzania hotelem. Nie zawiera modułu logowania, faktur, rezerwacji online ani integracji z zewnętrznymi serwisami. Program można jednak rozbudować w przyszłości o takie elementy.

3. Architektura i struktura aplikacji

3.1. Architektura aplikacji

Aplikacja została przygotowana jako program desktopowy WPF. Projekt wykorzystuje prostą architekturę warstwową z elementami wzorca MVVM. Interfejs użytkownika znajduje się w pliku XAML, logika prezentacji w klasie `MainViewModel`, a operacje biznesowe w serwisie `HotelService`.



Rysunek 3.1. Uproszczona architektura aplikacji `HotelManagementApp`

3.2. Podział projektu na warstwy i moduły

Projekt można podzielić na kilka części:

- warstwa modeli, czyli klasy opisujące dane w systemie,
- warstwa danych, czyli klasa `HotelDbContext` oraz migracje,
- warstwa repozytoriów, czyli klasy `IRepository` i `Repository`,
- warstwa usług, czyli `IHotelService` i `HotelService`,
- warstwa `ViewModel`, czyli `MainViewModel`,
- warstwa interfejsu, czyli `MainWindow.xaml` oraz `MainWindow.xaml.cs`.

3.3. Struktura folderów oraz organizacja kodu

Struktura projektu została podzielona według odpowiedzialności plików.

```
HotelManagementApp
|-- Commands
|   |-- RelayCommand.cs
|-- Data
|   |-- HotelDbContext.cs
|   |-- DbSeeder.cs
|-- Migrations
|   |-- 20260608000000_InitialCreate.cs
|   |-- HotelDbContextModelSnapshot.cs
|-- Models
|   |-- Guest.cs
|   |-- Room.cs
|   |-- RoomType.cs
|   |-- Reservation.cs
|   |-- Payment.cs
|   |-- Employee.cs
|   |-- Service.cs
```

```
| |-- ReservationService.cs
|-- Repositories
| |-- IRepository.cs
| |-- Repository.cs
|-- Services
| |-- IHotelService.cs
| |-- HotelService.cs
|-- ViewModels
| |-- MainViewModel.cs
| |-- ObservableObject.cs
|-- App.xaml.cs
|-- App.xaml
|-- MainWindow.xaml
|-- MainWindow.xaml.cs
|-- HotelManagementApp.csproj
```

Listing 3.1. Uproszczona struktura folderów projektu

Folder `Models` zawiera klasy mapowane na tabele w bazie danych. Folder `Data` zawiera konfigurację bazy danych i dane startowe. Folder `Services` zawiera logikę aplikacji, a folder `ViewModels` zawiera dane i komendy używane przez interfejs WPF.

3.4. Zastosowane wzorce projektowe

W projekcie zastosowano kilka prostych wzorców i mechanizmów organizacji kodu. Najważniejszym z nich jest MVVM, ponieważ widok korzysta z danych i komend udostępnionych przez `MainViewModel`. Zastosowano również Dependency Injection w pliku `App.xaml.cs`, dzięki czemu klasy nie tworzą ręcznie swoich zależności.

Dostęp do danych został częściowo oddzielony za pomocą wzorca Repository. Logika biznesowa znajduje się w serwisie `HotelService`. Dzięki temu kod odpowiedzialny za interfejs nie musi bezpośrednio wykonywać wszystkich operacji na bazie danych.

3.5. Mechanizmy walidacji danych

Walidacja danych została wykonana w serwisie. Program sprawdza między innymi, czy imię, nazwisko, numer telefonu i adres e-mail gościa nie są puste oraz czy adres e-mail zawiera znak @. Przy rezerwacji sprawdzane jest, czy data wymeldowania jest późniejsza niż data zameldowania oraz czy pokój jest dostępny w podanym terminie.

Dla usług hotelowych sprawdzana jest nazwa i cena. Nazwa nie może być pusta ani zbyt długa, a cena nie może być ujemna. Dzięki temu użytkownik nie może zapisać podstawowo błędnych danych.

3.6. Obsługa błędów i wyjątków

W projekcie zastosowano prostą obsługę błędów. Metody ViewModelu wywołują operacje serwisu wewnątrz bloków `try/catch`. Jeżeli wystąpi błąd, komunikat zostaje pokazany użytkownikowi w dolnej części okna.

Przykładowo, gdy użytkownik wybierze datę wymeldowania wcześniejszą niż data zameldowania, system nie zapisze rezerwacji i pokaże informację o błędzie. Podobnie aplikacja reaguje na próbę dodania płatności z kwotą mniejszą lub równą zero.

3.7. Czytelność i organizacja projektu

Nazwy klas odpowiadają elementom domeny hotelowej, na przykład `Guest`, `Room`, `Reservation`, `Payment` i `Service`. Dzięki temu łatwo zrozumieć, za co odpowiada dana część kodu. Projekt nie jest nadmiernie skomplikowany, ale spełnia wymagania dotyczące podziału na warstwy, obsługi bazy danych i organizacji interfejsu użytkownika.

4. Technologie wykorzystane w projekcie

4.1. Technologie interfejsu użytkownika

Do wykonania interfejsu użytkownika wykorzystano WPF. Technologia ta została wybrana, ponieważ dobrze nadaje się do tworzenia aplikacji desktopowych w C# i pozwala budować czytelne okna z tabelami, formularzami oraz zakładkami.

W projekcie WPF zostało użyte do przygotowania głównego okna aplikacji, panelu statystyk, formularzy, przycisków, tabel `DataGrid`, zakładek `TabControl` oraz prostego stylu wizualnego. Interfejs został przygotowany w spokojnych kolorach niebieskich i jasnych, aby przypominał panel administracyjny recepcji.

4.2. Technologie dostępu do danych

Do obsługi danych wykorzystano Entity Framework Core. W projekcie służy on do łączenia klas C# z tabelami w bazie danych. Dane można dodawać i pobierać za pomocą obiektów, bez ręcznego pisanie wielu zapytań SQL.

Jako baza danych została użyta SQLite. Plik bazy danych ma nazwę `hotel.db`. Jest to dobre rozwiązanie dla projektu studenckiego, ponieważ nie wymaga instalacji osobnego serwera bazy danych i łatwo przenieść projekt na inny komputer.

4.3. Dodatkowe biblioteki i narzędzia

W projekcie użyto pakietów Entity Framework Core potrzebnych do pracy z SQLite i migracjami. Najważniejsze pakiety to:

- `Microsoft.EntityFrameworkCore`,
- `Microsoft.EntityFrameworkCore.Sqlite`,
- `Microsoft.EntityFrameworkCore.Tools`,
- `Microsoft.EntityFrameworkCore.Design`,
- `Microsoft.Extensions.DependencyInjection`.

4.4. Uzasadnienie wyboru technologii

WPF został wybrany, ponieważ pozwala uzyskać więcej punktów za warstwę interfejsu niż Windows Forms i umożliwia wygodne tworzenie okien desktopowych. Entity Framework Core został wybrany, ponieważ jest wyżej punktowany niż ręczne ADO.NET i upraszcza pracę z bazą danych. SQLite został wybrany, ponieważ projekt można uruchomić lokalnie bez dodatkowej konfiguracji serwera.

Zastosowanie MVVM, Dependency Injection, repozytoriów i serwisów zwiększa czytelność projektu oraz ułatwia omówienie architektury podczas obrony.

5. Projekt bazy danych

5.1. Opis modelu danych

Baza danych została zaprojektowana odpowiednio do potrzeb aplikacji hotelowej. Model danych zawiera encje odpowiadające najważniejszym elementom pracy hotelu: gościom, pokojom, typom pokoi, rezerwacjom, płatnościom, pracownikom i usługom dodatkowym.

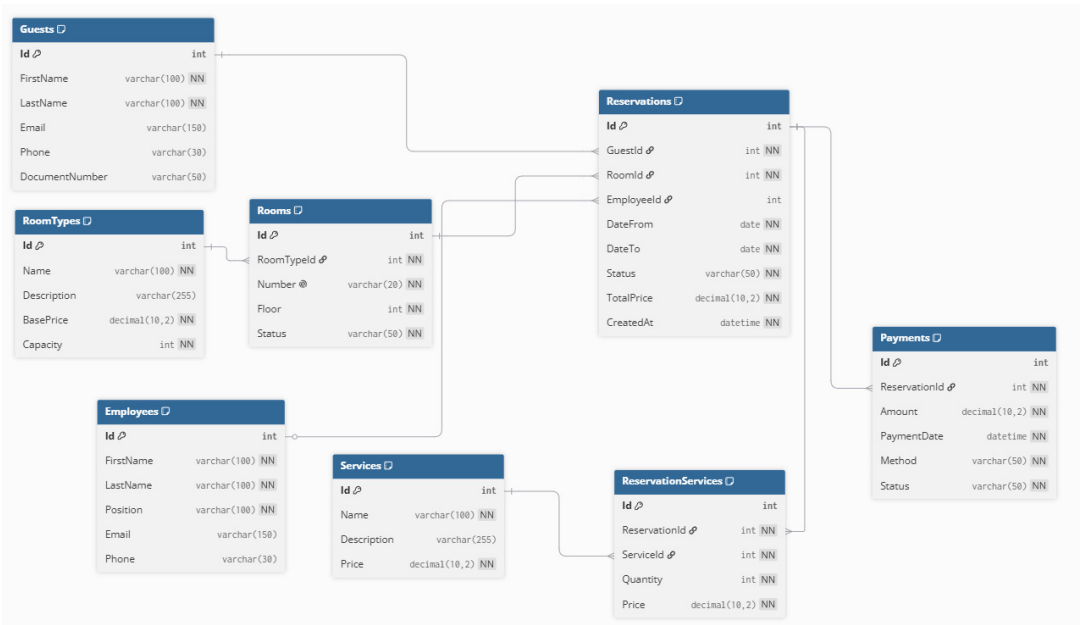
Tabela 5.1. Główne tabele w bazie danych

Tabela	Opis
Guests	Dane gości hotelowych.
Rooms	Pokoje hotelowe, numery i statusy.
RoomTypes	Typy pokoi i cena za noc.
Reservations	Rezerwacje z datą zameldowania i wymeldowania.
Payments	Płatności przypisane do rezerwacji.
Employees	Pracownicy obsługujący rezerwacje.
Services	Usługi dodatkowe, na przykład parking albo śniadanie.
ReservationServices	Tabela łącząca rezerwacje z usługami.

Taki model pozwala przechowywać dane w uporządkowany sposób i unikać powtarzania tych samych informacji w wielu miejscach.

5.2. Diagram ERD

Poniżej znajduje się uproszczony diagram ERD. Pokazuje on najważniejsze tabele oraz relacje między nimi.



Rysunek 5.1. Diagram ERD bazy danych aplikacji HotelManagementApp

5.3. Opis relacji i kluczy

Każda tabela posiada klucz główny w postaci pola `Id`. W projekcie zastosowano relacje jeden do wielu oraz jedną relację wiele do wielu z tabelą pośrednią.

Relacje w uproszczonej formie wyglądają tak:

```
RoomType 1 -> wiele Rooms
Guest 1 -> wiele Reservations
Room 1 -> wiele Reservations
Employee 1 -> wiele Reservations
Reservation 1 -> wiele Payments
Reservation wiele -> wiele Services przez ReservationServices
```

Listing 5.1. Relacje w bazie danych

Klucze obce zostały zapisane w klasach modeli, na przykład `RoomTypeId` w klasie `Room`, `GuestId` i `RoomId` w klasie `Reservation` oraz `ReservationId` w klasie `Payment`.

Fragment konfiguracji relacji w `HotelDbContext` pokazano w listingu poniżej.

```
1 modelBuilder.Entity<Room>()
2     .HasOne(r => r.RoomType)
3     .WithMany(rt => rt.Rooms)
4     .HasForeignKey(r => r.RoomTypeId)
5     .OnDelete(DeleteBehavior.Restrict);
6
7 modelBuilder.Entity<Reservation>()
8     .HasOne(r => r.Guest)
9     .WithMany(g => g.Reservations)
10    .HasForeignKey(r => r.GuestId)
11    .OnDelete(DeleteBehavior.Restrict);
12
13 modelBuilder.Entity<ReservationService>()
14    .HasKey(rs => new { rs.ReservationId, rs.ServiceId });
```

Listing 5.2. Fragment konfiguracji relacji w `HotelDbContext`

5.4. Informacje dotyczące normalizacji

Dane zostały rozdzielone na osobne tabele. Dane gościa są zapisane w tabeli `Guests`, dane pokoju w tabeli `Rooms`, a typ pokoju w tabeli `RoomTypes`. Dzięki temu cena za noc nie musi być powtarzana przy każdym pokoju, tylko znajduje się w typie pokoju.

Usługi hotelowe znajdują się w osobnej tabeli `Services`. Rezerwacja może mieć wiele usług, a jedna usługa może być używana w wielu rezerwacjach. Dlatego dodano tabelę pośrednią `ReservationServices`. Jest to poprawny sposób zapisania relacji wiele do wielu.

5.5. Opis migracji bazy danych

W projekcie użyto migracji Entity Framework Core. W folderze `Migrations` znajduje się migracja `InitialCreate`, która tworzy strukturę bazy danych. Przy uruchomieniu aplikacji wykonywana jest metoda `Database.Migrate()`, dzięki czemu baza danych jest tworzona lub aktualizowana automatycznie.

W przypadku ręcznego uruchamiania migracji można użyć poleceń:

```
dotnet ef migrations add InitialCreate  
dotnet ef database update
```

Listing 5.3. Polecenia migracji EF Core

5.6. Mechanizmy zapewniające integralność danych

Integralność danych jest zapewniona przez klucze główne, klucze obce i ograniczenia relacji. W konfiguracji zastosowano także unikalne indeksy, na przykład dla adresu e-mail gościa i numeru pokoju.

Dla części relacji ustawiono zachowanie `DeleteBehavior.Restrict`, aby nie usuwać przypadkowo danych powiązanych z rezerwacjami. Płatności są powiązane z rezerwacją i mogą być usunięte razem z nią, dlatego tam zastosowano kaskadowe usuwanie.

6. Implementacja aplikacji

6.1. Najważniejsze moduły systemu

Najważniejsze moduły systemu odpowiadają głównym częściom hotelu. Moduł gości służy do dodawania i przeglądania danych klientów. Moduł pokoi pokazuje numery pokoi, typy oraz statusy. Moduł rezerwacji pozwala wybrać gościa, pokój, pracownika i termin pobytu. Moduł płatności zapisuje wpłaty do rezerwacji. Moduł usług pozwala dodawać i edytować usługi dodatkowe.

6.2. Logika działania aplikacji

Po uruchomieniu aplikacja konfiguruje kontener Dependency Injection, tworzy połączenie z bazą SQLite i wykonuje migracje. Następnie dodawane są dane startowe, jeżeli baza jest pusta. Główne okno otrzymuje obiekt `MainViewModel`, który udostępnia kolekcje danych i komendy dla interfejsu.

```
1 var services = new ServiceCollection();
2 services.AddDbContext<HotelDbContext>(options =>
3     options.UseSqlite("Data Source=hotel.db"));
4
5 services.AddScoped(typeof(IRepository<>), typeof(Repository<>));
6 services.AddScoped<IHotelService, HotelService>();
7 services.AddTransient<MainViewModel>();
8 services.AddTransient<MainWindow>();
9
10 _serviceProvider = services.BuildServiceProvider();
11
12 using (var scope = _serviceProvider.CreateScope())
13 {
14     var db = scope.ServiceProvider.GetRequiredService<HotelDbContext>();
15     db.Database.Migrate();
16     DbSeeder.Seed(db);
17 }
```

Listing 6.1. Konfiguracja Dependency Injection i bazy SQLite

6.3. Realizacja operacji CRUD

Operacje CRUD zostały wykonane dla najważniejszych elementów systemu. Użytkownik może tworzyć nowe rekordy, wyświetlać je w tabelach oraz edytować wybrane usługi hotelowe. Część danych, na przykład pokoje i pracownicy, jest przygotowana jako dane startowe, aby aplikację można było od razu zaprezentować.

Poniżej pokazano skrócony przykład dodawania gościa. Nie jest to cały kod, tylko najważniejszy fragment odpowiedzialny za utworzenie obiektu i zapisanie go do bazy.

```
1 private async Task AddGuestAsync()
2 {
3     try
4     {
5         var guest = new Guest
6         {
```

```

7         FirstName = NewGuestFirstName,
8         LastName = NewGuestLastName,
9         Email = NewGuestEmail,
10        Phone = NewGuestPhone,
11        DocumentNumber = NewGuestDocument
12    };
13
14    await _hotelService.AddGuestAsync(guest);
15    await LoadAsync();
16    Message = "Dodano nowego gościa.";
17 }
18 catch (Exception ex)
19 {
20     Message = $"Nie można dodać gościa: {ex.Message}";
21 }
22 }

```

Listing 6.2. Przykład dodawania gościa w ViewModelu

Przy pobieraniu bardziej złożonych danych użyto metody `Include()`, aby pobrać również dane powiązane, na przykład gościa i pokój dla rezerwacji.

```

1 public Task<List<Reservation>> GetReservationsAsync() => _db.Reservations
2     .Include(r => r.Guest)
3     .Include(r => r.Room).ThenInclude(room => room!.RoomType)
4     .Include(r => r.Employee)
5     .AsNoTracking()
6     .OrderByDescending(r => r.CheckInDate)
7     .ToListAsync();

```

Listing 6.3. Przykład pobierania rezerwacji z danymi powiązаныmi

6.4. Dodawanie i edycja usług hotelowych

W poprawionej wersji projektu moduł usług umożliwia nie tylko przeglądanie, ale także dodawanie i edytowanie usług. Użytkownik może wybrać usługę z tabeli, zmienić jej nazwę lub cenę i zapisać zmiany. Jeżeli żadna usługa nie jest wybrana, formularz dodaje nowy rekord.

```

1 private async Task SaveServiceAsync()
2 {
3     var service = new Service
4     {
5         Id = SelectedService?.Id ?? 0,
6         Name = NewServiceName,
7         Price = NewServicePrice
8     };
9
10    if (SelectedService == null)
11        await _hotelService.AddServiceAsync(service);
12    else
13        await _hotelService.UpdateServiceAsync(service);
14
15    ClearServiceForm();
16    await LoadAsync();
17 }

```

Listing 6.4. Fragment zapisu nowej lub edytowanej usługi

6.5. Mechanizmy logowania i autoryzacji

W obecnej wersji projektu nie dodano logowania i autoryzacji. Aplikacja jest lokalnym programem desktopowym i zakłada się, że korzysta z niej pracownik recepcji hotelowej.

Gdyby projekt był rozwijany dalej, można byłoby dodać role użytkowników, na przykład administratora, recepcjonisty i menedżera. Administrator miałby pełny dostęp, a recepcjonista mógłby obsługiwać głównie rezerwacje i płatności.

6.6. Organizacja nawigacji

Nawigacja została zorganizowana przez zakładki w głównym oknie aplikacji. Użytkownik może przejść do panelu głównego, gości, pokoi, rezerwacji, płatności, pracowników i usług. Takie rozwiązanie jest proste i czytelne, ponieważ wszystkie moduły są dostępne z jednego miejsca.

```
1 <TabControl Margin="14">
2     <TabItem Header="Panel główny">
3         <!-- karty statystyk i opis modułów -->
4     </TabItem>
5     <TabItem Header="Goście" />
6     <TabItem Header="Pokoje" />
7     <TabItem Header="Rezerwacje" />
8     <TabItem Header="Płatności" />
9     <TabItem Header="Pracownicy" />
10    <TabItem Header="Usługi" />
11 </TabControl>
```

Listing 6.5. Fragment nawigacji w XAML

6.7. Interfejs użytkownika

Interfejs użytkownika składa się z górnego nagłówka, głównego obszaru z zakładkami i dolnego paska komunikatów. Panel główny zawiera karty statystyk i krótkie informacje o modułach. Pozostałe widoki mają podobny układ: tabela danych oraz formularz do wprowadzania danych.

6.8. Stylizacja i responsywność aplikacji

Aplikacja została wystylizowana w kolorach granatowym, niebieskim i jasnoszarym. Zastosowano karty z zaokrąglonymi rogami, czytelne nagłówki, wyróżnione przyciski i naprzemienne tło w tabelach. Program jest aplikacją desktopową, dlatego nie jest responsywny tak jak strona internetowa, ale ma ustawione minimalne wymiary okna i układ dopasowany do typowego monitora.

6.9. Walidacja danych w kodzie

Poniżej pokazano skrócony fragment walidacji. Program sprawdza wymagane pola gościa, poprawność dat rezerwacji oraz poprawność danych usługi.

```
1 private static void ValidateGuest(Guest guest)
2 {
3     if (string.IsNullOrEmpty(guest.FirstName) ||
4         string.IsNullOrEmpty(guest.LastName))
5         throw new InvalidOperationException("Imię i nazwisko są wymagane.");
6 }
```

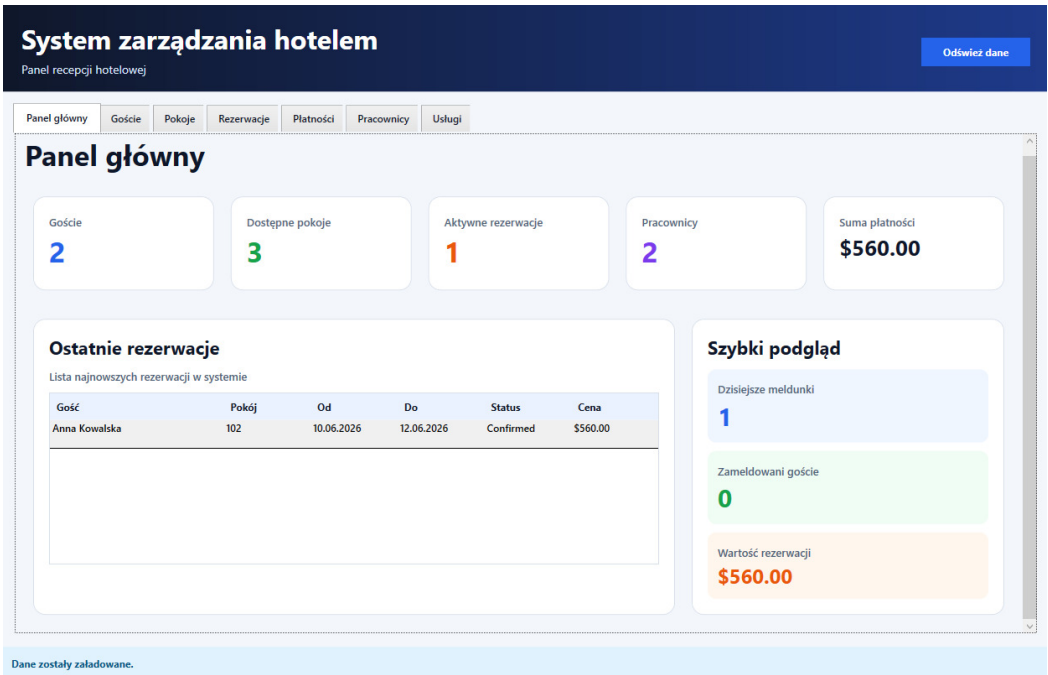
```
7      if (string.IsNullOrEmpty(guest.Email) || !guest.Email.Contains('@'))
8          throw new InvalidOperationException("Podaj poprawny adres e-mail.");
9
10     if (string.IsNullOrEmpty(guest.Phone))
11         throw new InvalidOperationException("Numer telefonu jest wymagany.");
12 }
13
14 private static void ValidateService(Service service)
15 {
16     if (string.IsNullOrEmpty(service.Name))
17         throw new InvalidOperationException("Nazwa usługi jest wymagana.");
18
19     if (service.Price < 0)
20         throw new InvalidOperationException("Cena usługi nie może być ujemna.");
21 }
```

Listing 6.6. Skrócony fragment walidacji danych w HotelService

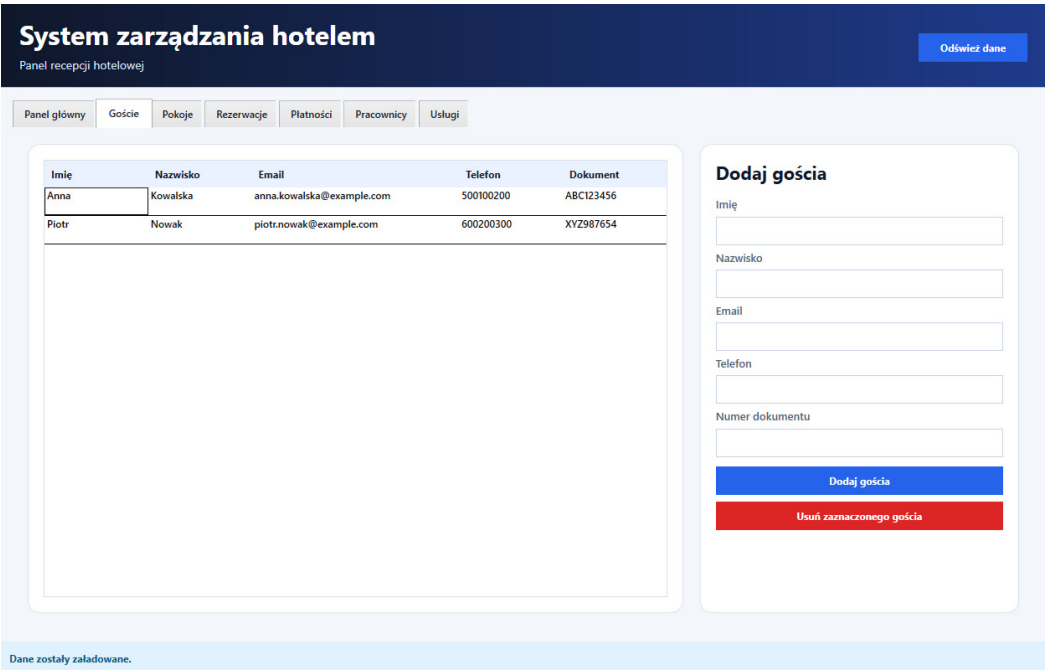
7. Prezentacja warstwy użytkowej projektu

7.1. Zrzuty ekranu aplikacji

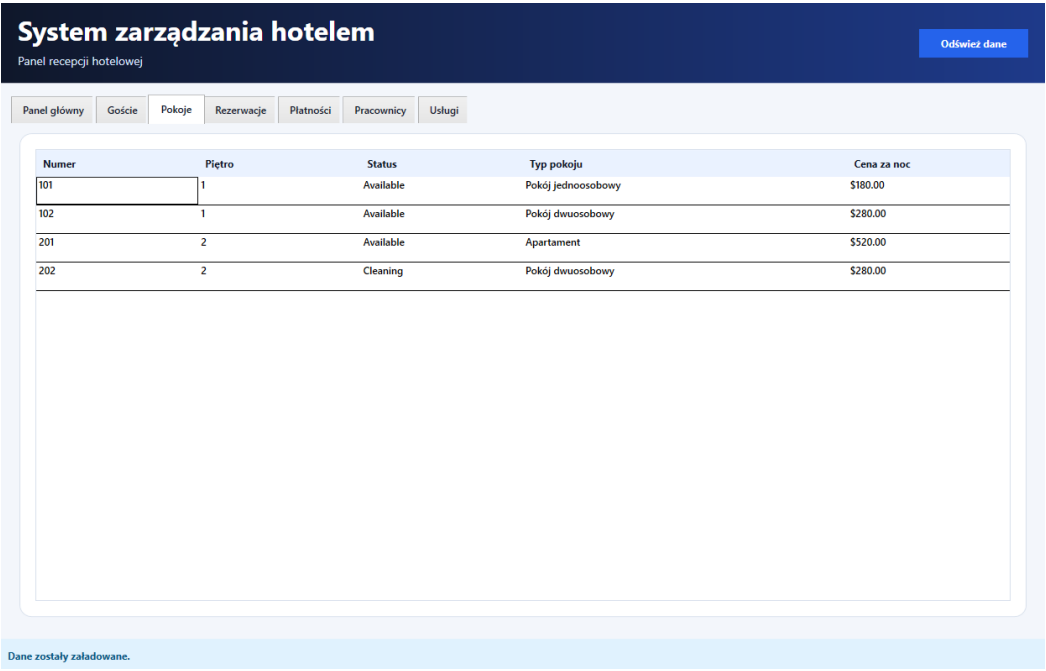
W tej części wstawiono zrzuty ekranu. Pokazują one najważniejsze widoki aplikacji i odpowiadają strukturze programu HotelManagementApp.



Rysunek 7.1. Ekran główny aplikacji z kartami statystyk i opisem modułów



Rysunek 7.2. Widok zarządzania gośćmi hotelowymi



Rysunek 7.3. Widok pokoi hotelowych z typem pokoju i statusem

System zarządzania hotelem

Panel recepcji hotelowej

[Odśwież dane](#)

[Panel główny](#)[Goście](#)[Pokoje](#)[Rezerwacje](#)[Płatności](#)[Pracownicy](#)[Usługi](#)

Gość	Pokój	Od	Do	Status	Cena
Anna Kowalska	102	10.06.2026	12.06.2026	Confirmed	\$560.00

Dodaj rezerwację

Gość

Pokój

Pracownik

Data od

Data do

[Sprawdź dostępność](#)

[Dodaj rezerwację](#)

[Usuń zaznaczoną rezerwację](#)

Dane zostały załadowane.

Rysunek 7.4. Widok tworzenia rezerwacji dla wybranego gościa i pokoju

System zarządzania hotelem

Panel recepcji hotelowej

[Odśwież dane](#)

[Panel główny](#)[Goście](#)[Pokoje](#)[Rezerwacje](#)[Płatności](#)[Pracownicy](#)[Usługi](#)

Rezerwacja	Gość	Kwota	Data	Metoda
1	Anna Kowalska	\$560.00	10.06.2026	Card

Dodaj płatność

Rezerwacja

Kwota

[Dodaj płatność](#)

[Usuń zaznaczoną płatność](#)

Dane zostały załadowane.

Rysunek 7.5. Widok płatności przypisanych do rezerwacji

System zarządzania hotelem
Panel recepcji hotelowej

Panel główny | Goście | Pokoje | Rezerwacje | Płatności | **Pracownicy** | Usługi

Imię	Nazwisko	Stanowisko
Maria	Winińska	Recepcjonistka
Tomasz	Zieliński	Manager

Dodaj pracownika

Imię:

Nazwisko:

Stanowisko:

Dodaj pracownika

Wyczyść formularz

Usuń zaznaczonego pracownika

Aby usunąć pracownika, najpierw zaznacz go na liście.

Dane zostały załadowane.

Rysunek 7.6. Widok pracowników hotelu

System zarządzania hotelem
Panel recepcji hotelowej

Panel główny | Goście | Pokoje | Rezerwacje | Płatności | **Pracownicy** | **Usługi**

Oferta usług dodatkowych

Kliknij usługę na liście, aby przenieść ją do formularza i edytować.

Nazwa	Cena
Parking	\$35.00
Spa	\$120.00
Śniadanie	\$45.00

Dodaj lub edytuj usługę

Nazwa usługi:

Cena:

Zapisz usługę

Wyczyść formularz

Usuń zaznaczoną usługę

Po zaznaczeniu usługi z listy formularz działa jako edycja. Jeżeli nic nie jest zaznaczone, system dodaje nową usługę.

Dane zostały załadowane.

Rysunek 7.7. Widok usług z możliwością dodawania i edycji

7.2. Opis działania najważniejszych funkcjonalności

Najważniejszą częścią aplikacji jest możliwość zarządzania rezerwacjami hotelowymi. Użytkownik wybiera gościa, pokój i termin pobytu. Program sprawdza, czy pokój jest dostępny, a następnie oblicza cenę na podstawie typu pokoju i liczby nocy.

Drugą ważną funkcją jest obsługa płatności. Użytkownik wybiera rezerwację, wpisuje kwotę i zapisuje płatność. Dane są widoczne w tabeli płatności oraz wpływają na statystyki na panelu głównym.

Kolejna funkcja to moduł usług. Użytkownik może dodać nową usługę, na przykład parking albo śniadanie, oraz edytować istniejącą usługę. Formularz został przygotowany tak, aby jedna sekcja służyła do dodawania i edycji.

7.3. Prezentacja interfejsu użytkownika

Interfejs został podzielony na trzy główne części. Na górze znajduje się nagłówek z nazwą systemu i przyciskiem odświeżenia danych. W środku znajduje się obszar roboczy z zakładkami. Komunikaty o błędach i wykonanych operacjach są pokazywane w formularzach lub w aktywnej sekcji programu, zależnie od wykonywanej operacji.

Panel główny pokazuje najważniejsze statystyki: aktywne rezerwacje, liczbę gości, dostępne pokoje, liczbę usług i wartość płatności. Dzięki temu użytkownik po uruchomieniu programu od razu widzi aktualny stan systemu.

7.4. Przykładowe scenariusze działania aplikacji

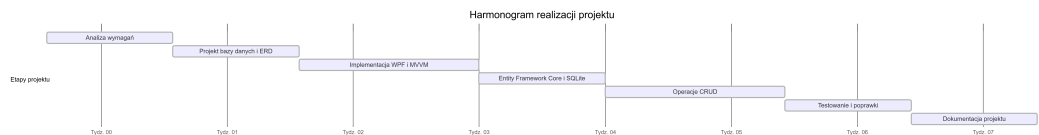
Pierwszy scenariusz dotyczy dodania gościa. Użytkownik przechodzi do widoku gości, wpisuje imię, nazwisko, telefon, adres e-mail i numer dokumentu, a następnie naciska przycisk dodania. Jeżeli dane są poprawne, gość pojawia się w tabeli.

Drugi scenariusz dotyczy utworzenia rezerwacji. Użytkownik wybiera gościa, pokój, pracownika oraz datę zameldowania i wymeldowania. Po zapisaniu rezerwacja pojawia się na liście. Jeżeli pokój jest zajęty w podanym terminie, aplikacja pokazuje komunikat o braku dostępności.

Trzeci scenariusz dotyczy edycji usługi. Użytkownik przechodzi do widoku usług, wybiera usługę z tabeli, zmienia nazwę lub cenę i naciska przycisk zapisu. System zapisuje zmiany w bazie danych.

8. Harmonogram realizacji projektu

Projekt można było wykonać w ciągu jednego tygodnia. Prace zostały rozdzielone na kilka prostych etapów. Najpierw wybrano temat i określono zakres funkcji. Następnie przygotowano modele danych, bazę, migracje i dane startowe. Później wykonano interfejs, obsługę operacji CRUD, walidację oraz końcowe poprawki wyglądu.



Rysunek 8.1. Orientacyjny harmonogram realizacji projektu w ciągu tygodnia

Tabela 8.1. Etapy realizacji projektu

Etap	Opis prac
Dzień 1	Wybór tematu, analiza wymagań i zaplanowanie tabel.
Dzień 2	Utworzenie projektu WPF, modeli i kontekstu EF Core.
Dzień 3	Przygotowanie migracji, repozytoriów, serwisów i danych startowych.
Dzień 4	Wykonanie ViewModelu, komend i podstawowej nawigacji.
Dzień 5	Dodanie operacji CRUD, płatności i dostępności pokoi.
Dzień 6	Poprawa panelu głównego, modułu usług i stylizacji.
Dzień 7	Testowanie, poprawki, przygotowanie dokumentacji.

Harmonogram jest orientacyjny. W praktyce część zadań mogła być wykonywana równolegle, na przykład poprawki interfejsu i poprawki walidacji danych.

9. Instrukcja uruchomienia projektu

9.1. Wymagania systemowe

Do uruchomienia projektu potrzebny jest komputer z systemem Windows. Projekt najlepiej uruchomić w Visual Studio 2022. W systemie powinien być zainstalowany .NET SDK oraz składnik .NET desktop development. Nie trzeba instalować osobnego serwera bazy danych, ponieważ projekt korzysta z SQLite.

9.2. Sposób konfiguracji środowiska

Najpierw należy zainstalować Visual Studio 2022. Podczas instalacji trzeba zaznaczyć składnik .NET desktop development. Następnie należy rozpakować projekt i otworzyć plik `HotelManagementApp.csproj` albo całe rozwiązanie, jeżeli zostało dodane.

Po otwarciu projektu Visual Studio powinno automatycznie przywrócić pakiety NuGet. Jeżeli tego nie zrobi, należy otworzyć terminal w folderze projektu i wpisać:

```
dotnet restore
```

Listing 9.1. Przywrócenie pakietów NuGet

9.3. Instrukcja uruchomienia aplikacji

Aby uruchomić projekt od zera, należy wykonać następujące kroki:

1. Rozpakować archiwum z projektem.
2. Otworzyć folder `HotelManagementApp`.
3. Uruchomić plik `HotelManagementApp.csproj` w Visual Studio.
4. Poczekać, aż zostaną przywrócone pakiety NuGet.
5. Wybrać projekt `HotelManagementApp` jako projekt startowy.
6. Nacisnąć F5 albo przycisk Start.
7. Po uruchomieniu aplikacji można korzystać z zakładek w głównym oknie.

Jeżeli pojawi się problem z pakietami, należy wykonać polecenie `dotnet restore`. Jeżeli pojawi się problem z bazą danych, można usunąć plik `hotel.db` i uruchomić projekt ponownie.

9.4. Informacje dotyczące bazy danych

Aplikacja korzysta z SQLite. Nazwa pliku bazy danych to `hotel.db`. Baza jest tworzona i aktualizowana automatycznie przy uruchomieniu programu przez metodę `Database.Migrate()`.

Połączenie z bazą znajduje się w pliku `App.xaml.cs`. W projekcie użyto connection stringa:

```
Data Source=hotel.db
```

Listing 9.2. Connection string SQLite

9.5. Dane logowania testowego

W obecnej wersji aplikacja nie posiada logowania, dlatego nie ma danych logowania testowego. Program uruchamia się od razu po włączeniu i użytkownik ma dostęp do wszystkich modułów.

9.6. Miejsce na repozytorium GitHub

W tym miejscu należy wkleić link do repozytorium GitHub z projektem:

```
https://github.com/Freezzz/HotelManagementApp/tree/main
```

Listing 9.3. Miejsce na link do repozytorium GitHub

10. Podsumowanie

10.1. Ocena stopnia realizacji założeń projektowych

Projekt zrealizował główne założenia dla tematu *System zarządzania hotelem*. Aplikacja pozwala na podstawowe zarządzanie gośćmi, pokojami, rezerwacjami, płatnościami, pracownikami i usługami dodatkowymi. Został wykonany interfejs graficzny, lokalna baza danych SQLite, migracje Entity Framework Core oraz prosta walidacja danych.

Program nie jest pełnym systemem hotelowym klasy komercyjnej, ale nie taki był główny zakres projektu. Dla podstawowego systemu zarządzania wykonano najważniejsze funkcje administracyjne, które pozwalają kontrolować dane recepcji hotelowej.

10.2. Napotkane problemy

Podczas realizacji projektu mogły pojawić się typowe problemy związane z połączeniem aplikacji WPF z bazą danych. Jednym z problemów była poprawna konfiguracja Entity Framework Core i SQLite, aby baza tworzyła się automatycznie przy uruchomieniu programu.

Drugim problemem było sprawdzanie dostępności pokoi. Trzeba było uwzględnić sytuację, w której dwie rezerwacje nakładają się na siebie czasowo. W tym celu w serwisie sprawdzany jest konflikt dat dla wybranego pokoju.

Kolejnym problemem było ustawienie interfejsu tak, aby panel główny nie wyglądał zbyt pusto. Dlatego dodano karty statystyk, opis modułów oraz poprawiono wygląd zakładki usług.

10.3. Możliwe kierunki dalszego rozwoju aplikacji

Projekt można rozbudować do pełniejszego systemu dla hotelu. Można dodać logowanie i role użytkowników, na przykład administratora, recepcjonisty i menedżera. Można też dodać faktury, raporty miesięczne, eksport do pliku PDF, historię zmian oraz pełne zarządzanie pokojami.

W przyszłości można również dodać moduł rezerwacji online albo integrację z zewnętrznymi portalami. Obecna wersja jest jednak podstawowym systemem zarządzania i realizuje zakres przyjęty w projekcie.

Spis rysunków

3.1	Uproszczona architektura aplikacji HotelManagementApp	13
5.1	Diagram ERD bazy danych aplikacji HotelManagementApp	17
7.1	Ekran główny aplikacji z kartami statystyk i opisem modułów	24
7.2	Widok zarządzania gośćmi hotelowymi	25
7.3	Widok pokoi hotelowych z typem pokoju i statusem	25
7.4	Widok tworzenia rezerwacji dla wybranego gościa i pokoju	26
7.5	Widok płatności przypisanych do rezerwacji	26
7.6	Widok pracowników hotelu	27
7.7	Widok usług z możliwością dodawania i edycji	27
8.1	Orientacyjny harmonogram realizacji projektu w ciągu tygodnia	29

Spis tabel

2.1	Najważniejsze wymagania funkcjonalne aplikacji	11
5.1	Główne tabele w bazie danych	17
8.1	Etapy realizacji projektu	29

Spis listingów

3.1	Uproszczona struktura folderów projektu	13
5.1	Relacje w bazie danych	18
5.2	Fragment konfiguracji relacji w HotelDbContext	18
5.3	Polecenia migracji EF Core	19
6.1	Konfiguracja Dependency Injection i bazy SQLite	20
6.2	Przykład dodawania gościa w ViewModelu	20
6.3	Przykład pobierania rezerwacji z danymi powiązanymi	21
6.4	Fragment zapisu nowej lub edytowanej usługi	21
6.5	Fragment nawigacji w XAML	22
6.6	Skrócony fragment walidacji danych w HotelService	22
9.1	Przywrócenie pakietów NuGet	30
9.2	Connection string SQLite	30
9.3	Miejsce na link do repozytorium GitHub	31